

Магистратура
«Искусственный интеллект»

Инструменты разработки
Лекция 1

Чернов Евгений
НИУ ВШЭ

Linux и командная строка

Как устроена ОС, на которой работает вся ML-инфраструктура, и как уверенно управлять ей из терминала

План занятия

1. **Зачем Linux и как он устроен**
   ядро, userspace, shell, анатомия команды
2. **Файловая система**
   дерево от /, пути, FHS, inode и ссылки
3. **Права доступа**
   rwx, chmod, запуск скриптов, sudo
4. **Процессы**
   PID, дерево процессов, сигналы, фон
5. **Потоки и конвейеры**
   stdin, stdout, stderr, редиректы и |
6. **Инструменты**
   grep, find, sed, awk, PATH, apt

Зачем ML-инженеру Linux

Учитесь вы, скорее всего, на Windows или macOS, а модель обучается и работает на сервере. Сервер почти всегда Linux — и часто без рабочего стола: есть только терминал.

удалённый сервер

```
$ ssh ivan@gpu-server
$ uname -sr
Linux 6.8.0-45-generic
$ head -1 /etc/os-release
PRETTY_NAME="Ubuntu 24.04.1 LTS"
$ nvidia-smi -L
GPU 0: NVIDIA A100-SXM4-80GB (UUID: GPU-…)
# весь ML-стек живёт здесь
```

Серверы и облака
AWS, GCP, GPU-кластеры — почти всегда Linux

Родная среда ML-стека
CUDA, PyTorch и Docker работают здесь

Всё через команды
нет мышки — зато всё можно автоматизировать

Понимать, а не копировать
LLM напишет команду — прочитать её должны вы

! **Где взять Linux:** WSL на Windows, виртуалка (UTM, VirtualBox) или дешёвый VPS. macOS — тоже Unix: команды почти те же. Ломайте свою машину, а не прод.

01

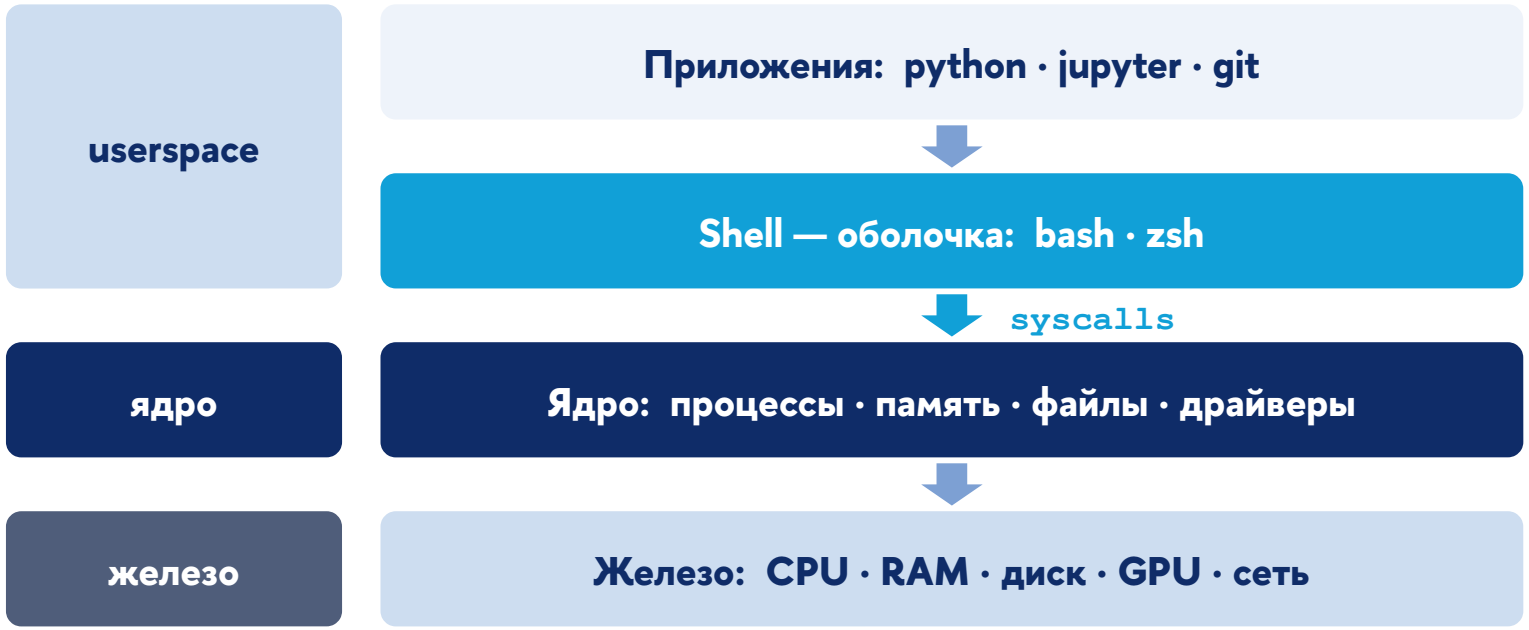
РАЗДЕЛ 01

Система и файлы

Куда уходит набранная команда и как устроено единое дерево файлов

Ядро, userspace и оболочка

Вы набираете команду в оболочке, оболочка запускает программу, программа просит ядро через системные вызовы — и только ядро работает с диском, памятью и сетью.



Shell (оболочка)

программа, которая читает набранную строку и запускает другие программы. Чаще всего bash или zsh

Ядро (kernel)

единственная часть ОС с прямым доступом к железу: решает, кому дать память, диск и сеть

Анатомия команды: что, как и над чем

`ls` — команда — что делать

`-la` — опции — как делать

`/home/ivan` — аргумент — над чем

терминал

```
$ ls                    # без опций
$ ls -l ~/project       # опция + аргумент
$ cp -r src/ backup/    # два аргумента
$ grep -n TODO app.py   # опция, шаблон, файл
$ ls --help             # краткая справка
$ man ls                # руководство, выход — q
```

`-l -a = -la`
короткие опции можно склеивать

`--all = -a`
длинная форма — понятнее в скриптах

`--help · man`
справка есть почти у каждой команды

Где я? Пути и перемещение по дереву

Первая команда в незнакомой системе — «где я?». Абсолютный путь начинается от корня /, относительный — от текущей папки.

терминал

```
$ pwd
/home/ivan
$ cd project/src     # относительный путь
$ cd /etc            # абсолютный путь
$ cd ~               # домой (или просто cd)
$ cd ..              # на уровень вверх: /home
$ cd -               # назад, в прошлую папку
/home/ivan
$ ls ./run.sh        # файл в текущей папке
```

/
корень всего дерева

~
домашняя папка: /home/ivan

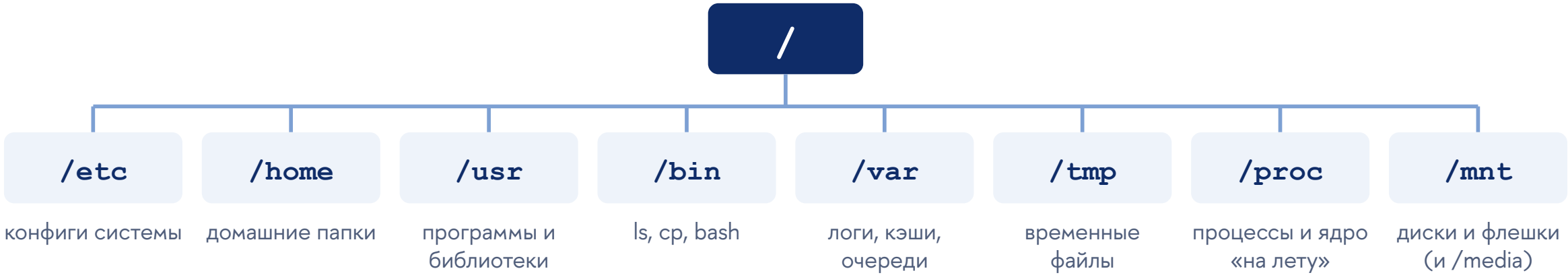
.
текущая папка

..
родительская папка

Tab
дополнить имя — жмите чаще

FHS: одно дерево от корня /

В Linux нет дисков C: и D:. Всё — одно дерево от корня /, а флешки и диски монтируются внутрь него как обычные папки. Стандарт FHS задаёт, что где лежит.



В Ubuntu 24.04 /bin — ссылка на /usr/bin: `ls -ld /bin` покажет /bin -> usr/bin. Двух разных /home быть не может: корень один.

Файлы руками: создать, скопировать, удалить

терминал

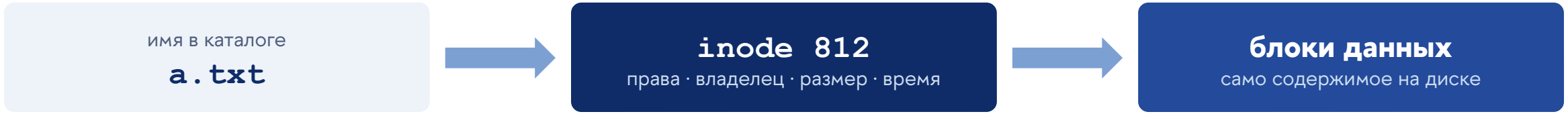
```
$ mkdir -p ~/devtools/src    # -p: с родителями
$ cd ~/devtools
$ touch notes.txt            # пустой файл
$ echo "print('hi')" > src/app.py
$ cp src/app.py src/old.py   # копия
$ mv notes.txt .notes.txt    # переименовать
$ ls -a
.  ..  .notes.txt  src
$ python3 src/app.py
hi
$ rm -r src                  # корзины нет!
```

mkdir	создать папку; -p — всю цепочку
touch	пустой файл или обновить время
cp	копировать; -r — папку целиком
mv	переместить или переименовать
rm	удалить; -r — папку, -f — без вопросов
cat	вывести содержимое файла

Имя с точки (.notes.txt) — скрытый файл: виден только с **ls -a**

inode: имя файла ≠ его содержимое

Паспорт файла — inode: владелец, права, размер, время и адреса блоков на диске. Имени в inode нет: имя лежит в каталоге и указывает на номер inode.



терминал

```
$ ln a.txt hard          # жёсткая ссылка
$ ln -s a.txt soft       # символьная
$ ls -li
812 -rw-r--r-- 2 ivan ml 0 Oct 20 09:03 a.txt
812 -rw-r--r-- 2 ivan ml 0 Oct 20 09:03 hard
813 lrwxrwxrwx 1 ivan ml 5 Oct 20 09:03 soft -> a.txt
```

Жёсткая ссылка · ln

второе имя того же inode. rm удаляет имя, а данные живут, пока есть хоть одно имя

Символьная ссылка · ln -s

отдельный файл с путём к цели; может вести на папку; «повиснет», если цель удалить

02

РАЗДЕЛ 02

Права и процессы

Кто может читать и запускать файлы и как управлять запущенными программами

Как читать строку ls -l

`-rw-r--r--`	`1`	`ivan`	`ml`	`2048`	`Oct 20 09:03`	`model.py`
тип и права	ссылки	владелец	группа	размер, байт	дата изменения	имя

терминал

```
$ ls -l
total 12
drwxr-xr-x 2 ivan ml 4096 Oct 20 09:03 data
-rw-r--r-- 1 ivan ml 2048 Oct 20 09:03 model.py
drwxr-xr-x 3 ivan ml 4096 Oct 20 09:03 project
$ ls -lah     # + скрытые, размеры 4.0K, 2.0K
```

Первый символ — тип файла

- `-` обычный файл
- `d` каталог
- `l` символьная ссылка
- `b` `c` устройства: диски, терминалы

Дальше 9 символов — права rwx для владельца, группы и остальных

Права: три действия для трёх аудиторий



`r = 4` `w = 2` `x = 1` → rwx = 7, r-x = 5, r-- = 4: итого 754

	для файла	для каталога
r	читать содержимое	видеть список имён (ls)
w	менять содержимое	создавать и удалять имена
x	запускать как программу	входить (cd) и идти по пути

терминал

```
$ ls -l run.sh
-rw-r--r-- 1 ivan ml 120 Oct 20 09:03 run.sh
$ chmod +x run.sh      # +x всем трём
$ ls -l run.sh
-rwxr-xr-x 1 ivan ml 120 Oct 20 09:03 run.sh
$ chmod u+x run.sh     # только владельцу
$ chmod 640 key.pem    # rw- r-- ---
$ chmod 755 run.sh     # rwx r-x r-x
$ chmod 600 .env       # только я
# сменить владельца и группу
$ sudo chown ivan:ml data/
```

Запуск ./файл: бит x и shebang

терминал

```
$ cat hello.sh
#!/bin/sh
echo hello-dev-tools
$ ./hello.sh
bash: ./hello.sh: Permission denied
$ sh hello.sh      # интерпретатор читает файл
hello-dev-tools
$ chmod +x hello.sh
$ ./hello.sh
hello-dev-tools
```

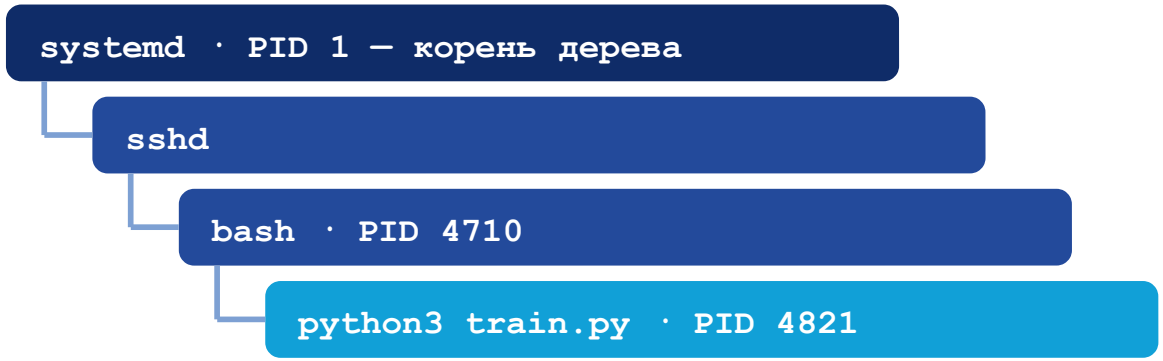
`#!/usr/bin/env python3`
shebang: первая строка говорит, чем запускать файл

`python3 app.py`
запускается python3 — файлу бит x не нужен

`./app.py`
запускается сам файл — нужны бит x и shebang

`root` — суперпользователь (UID 0), ему можно почти всё. **`sudo команда`** выполняет одну команду от root: семь раз прочитайте, один раз запустите.

Процессы: PID, родители и сигналы



Сигнал	Как послать	Что делает
SIGINT	Ctrl+C	прервать текущую команду
SIGTERM	kill PID	попросить завершиться
SIGKILL	kill -9 PID	убить сразу, без уборки
SIGTSTP	Ctrl+Z	пауза; продолжить: fg / bg

терминал

```
$ python3 train.py &       # в фон
[1] 4821
$ ps -o pid,ppid,cmd
    PID    PPID CMD
   4710    4709 -bash
   4821    4710 python3 train.py
   4830    4710 ps -o pid,ppid,cmd
$ ps aux | grep train      # все процессы
$ kill 4821                # SIGTERM
$ kill -9 4821             # SIGKILL
```

03

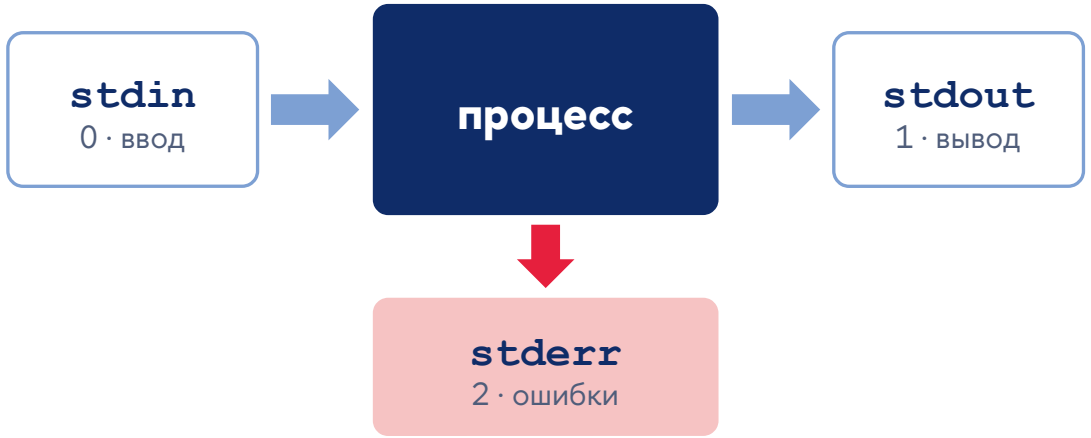
РАЗДЕЛ 03

Потоки и инструменты

Конвейеры из маленьких утилит: поиск, обработка текста, окружение и пакеты

Три потока: stdin, stdout, stderr

У каждого процесса три стандартных потока со своими номерами. По умолчанию все они подключены к терминалу, но любой можно перенаправить в файл.



редиректы

```
$ ls -lah > files.txt      # stdout в файл
$ echo done >> files.txt   # дописать в конец
$ cat nope 2> err.log      # stderr в файл
$ sort < names.txt         # stdin из файла
$ cmd > all.log 2>&1       # оба потока в файл
$ cmd &> all.log           # то же, короче (bash)
```

! **`>` перезаписывает файл без вопросов.** Порядок важен: в `cmd 2>&1 > f` ошибки уйдут на экран, а не в файл.

Конвейер: stdout одной команды — stdin следующей

```
$ cat app.log | grep ERROR | wc -l
```

cat app.log → **grep ERROR** → **wc -l**

1 · все строки

```
INFO ok
ERROR disk
INFO ok
ERROR net
```

2 · только ERROR

```
ERROR disk
ERROR net
```

3 · сколько строк

```
2
```

! | не создаёт промежуточных файлов — данные текут через память. Эту задачу можно решить и короче: **`grep -c ERROR app.log`**

Смотрим и правим файлы, не выходя из терминала

На сервере обычно нет VS Code. Чтобы посмотреть файл, хватает cat, less, head и tail; чтобы поправить строку — nano или vim.

просмотр

```
$ cat app.py            # весь файл сразу
$ less huge.log         # листать; выход — q
$ head -n 20 app.log    # первые 20 строк
$ tail -n 20 app.log    # последние 20 строк
$ tail -f app.log       # следить за ростом
$ nano config.yaml      # простой редактор
```

vim: минимум, чтобы выжить

1. `vim app.py` — открыть файл
2. `i` — режим вставки, внизу INSERT
3. `Esc` — назад в режим команд
4. `:wq` — записать и выйти
5. `:q!` — выйти без сохранения

Поиск: grep ищет строки, find — файлы

grep смотрит внутрь файлов и печатает подходящие строки. find обходит дерево папок и находит сами файлы — по имени, типу, размеру и времени.

grep — строки в файлах

```
$ grep "error" app.log
$ grep -in "todo" *.py    # без регистра, с №
$ grep -v "DEBUG" app.log # кроме DEBUG
$ grep -c "ERROR" app.log # число строк
$ grep -rn "API_KEY" src/ # по всей папке
```

find — файлы в дереве

```
$ find . -name '*.py'          # по маске
$ find . -type d               # только папки
$ find . -type f -size +100M # > 100 МБ
$ find . -mtime -1             # за сутки
$ find /var/log -name '*.log'
```

Вместе: `find . -name '*.py' -exec grep -l 'def train' {} +` — find находит файлы, grep ищет внутри них.

sed и awk: обработка текста потоком

Оба читают текст построчно. sed — про строки и замены, awk — про колонки (поля) и простую арифметику. Пример — тот же app.log.

sed — редактор потока

```
$ sed 's/ERROR/ERR/g' app.log # замена
$ sed -n '2,3p' app.log       # строки 2–3
$ sed '/INFO/d' app.log       # убрать INFO
$ sed -i 's/8000/8080/' conf  # в самом файле
```

awk — работа с колонками

```
$ awk '{print $1}' app.log    # 1-я колонка
$ awk '$1=="ERROR" {print $2}' app.log
disk
net
$ awk 'END {print NR}' app.log # строк
4
```

$0 — вся строка, **$1**, **$2**… — поля через пробел (другой разделитель: **-F:**). **sed -i** меняет файл на месте — сначала запустите без -i.

Переменные окружения и PATH

терминал

```
$ echo $HOME
/home/ivan
$ echo $PATH
/usr/local/bin:/usr/bin:/bin:…
$ which python3
/usr/bin/python3
$ X=1;        bash -c 'echo "X=$X"'
X=
$ export X=1; bash -c 'echo "X=$X"'
X=1
$ export PATH="$HOME/bin:$PATH"
```

PATH
папки через «:»; команда ищется слева направо, берётся первая

export
передать переменную дочерним процессам

~/.bashrc
выполняется при старте bash: export, alias, PATH

env
показать все переменные окружения

! **Секреты — не в код и не в git:** токены передают через переменные окружения или файл .env, добавленный в .gitignore.

Пакеты: apt для системы, pip для проекта

Программы в Linux ставят не с сайтов, а из репозиториев дистрибутива — менеджером пакетов. В Ubuntu и Debian это apt.

терминал

```
$ sudo apt update          # обновить индекс
$ sudo apt install htop    # поставить
$ sudo apt upgrade         # обновить пакеты
$ apt search postgres      # найти
$ sudo apt remove htop     # удалить
```

	apt	pip · uv · conda
Что ставит	программы ОС: git, htop, nginx	библиотеки Python: pandas, torch
Куда	в систему, нужен sudo	в виртуальное окружение
Аналоги	dnf (Fedora), brew (macOS)	poetry, pipx

! **`apt update`** обновляет только список пакетов. В Ubuntu 24.04 **`pip install`** вне виртуального окружения выдаст **`externally-managed-environment`** — это подсказка создать venv.

Горячие клавиши и первые ошибки

Tab	дополнить имя
↑ ↓	история команд
Ctrl+R	поиск по истории
Ctrl+C	прервать команду
Ctrl+D	конец ввода / выход
Ctrl+L	очистить экран
q	выйти из less и man

`command not found`
опечатка или программа не установлена: проверьте which

`No such file or directory`
не тот путь: pwd, ls и дополнение по Tab

`Permission denied`
не хватает прав: ls -l, затем chmod или осознанный sudo

«терминал завис»
команда ждёт ввода: Ctrl+C прервёт, Ctrl+D закроет ввод

ИТОГИ ЛЕКЦИИ

Что важно унести

1 **Одно дерево от /**
пути и FHS; имя → inode → данные

2 **Права и root**
rwx для u, g, o; chmod; sudo — осознанно

3 **Процессы и сигналы**
PID и родители; SIGTERM раньше SIGKILL

4 **Потоки и конвейеры**
> >> 2> и | собирают утилиты в цепочки

Дальше в курсе — **Git: основы и устройство**: как версионировать код и не терять работу.


ВЫСШАЯ • ШКОЛА • ЭКОНОМИКИ •